

M002 Extension 5 — Redstone AND Solver

Topic: All 5 Redstone AND Rules + Single Checks · **Course:** Maze Madness · **Level:** Extension (Advanced) · **Time:** about 60 minutes

This is an **extension challenge** for students who finished Week 3. It uses the **same Week 3 world**. Your target is the **hardest maze** — the big one **your teacher will show you a picture of**: tall clear-glass frames packed with red redstone and a diamond block at the end. Walk the world and find it. This time you write the **whole solver** — all five **AND** rules at once, plus the single checks. Every choice reads **redstone**.

The redstone contract

Single checks — one redstone: - RS **left** → turn right · RS **right** → turn left - RS **below** → move up 1 · RS **above** → move down 1

AND rules — two redstones, **both** true: - **#1** RS left **and** below → up 1, turn right - **#2** RS right **and** below → up 1, turn left - **#3** RS ahead **and** below → up 1, turn around - **#4** RS left **and** right → up 5 - **#5** RS above **and** below → back 2, turn right, forward 2

Chat words: **l** turn left · **r** turn right · **run** start the solver · **r1** teleport the agent back to you.

1 · Read the Chat Commands

Each block binds a **chat word** to an action. Type the word, the agent runs the code.

```
def turn_left():
    agent.turn(LEFT_TURN)
player.on_chat("l", turn_left)
```

What does the agent do when you type **l** ?


```
def go_back():
    agent.teleport_to_player()
player.on_chat("r1", go_back)
```

You type `r1` . Where does the agent go? When does that help in a long maze?

2 · Trace the Solver 🔍

This loop runs when you type `run`. It checks the **AND** rules first, then moves forward.

```
while True:
    if agent.detect(REDSTONE, LEFT) and agent.detect(REDSTONE, RIGHT):
        agent.move(UP, 5)
    elif agent.detect(REDSTONE, UP) and agent.detect(REDSTONE, DOWN):
        agent.move(BACK, 2)
        agent.turn(RIGHT_TURN)
        agent.move(FORWARD, 2)
    elif agent.detect(REDSTONE, FORWARD) and agent.detect(REDSTONE, DOWN):
        agent.move(UP, 1)
        agent.turn(RIGHT_TURN)
        agent.turn(RIGHT_TURN)
    elif agent.detect(REDSTONE, LEFT) and agent.detect(REDSTONE, DOWN):
        agent.move(UP, 1)
        agent.turn(RIGHT_TURN)
    agent.move(FORWARD, 1)
```

Which AND rule number is each `if` / `elif` — #1 to #5? One is missing. Which?

Rule #4 is `RS LEFT and RS RIGHT`. What single move does it run, and why is no turn needed?

Rule #1 is `RS LEFT and RS DOWN`. A tile has redstone on the left only — no redstone below. Walk it through the code. Does #1 run? What happens instead?

Why are all the AND checks first, before any single check? What would rule #1 lose if `if agent.detect(REDSTONE, DOWN): agent.move(UP, 1)` was checked first?

3 · Spot the Bug 🐛

Each block is broken. Read its goal, rewrite it, then explain the fix.

Bug A — Rule #1: turn right **only when redstone is left AND below**. This checks redstone left only, so it turns on a plain left marker that has no redstone below it.

```
if agent.detect(REDSTONE, LEFT):  
    agent.move(UP, 1)  
    agent.turn(RIGHT_TURN)  
agent.move(FORWARD, 1)
```

Fixed code:

Why was it wrong?

Bug B — Rule #5 needs redstone **above AND below** — both. This uses **or**, so redstone below alone fires it and the agent jumps backward at the wrong tile.

```
if agent.detect(REDSTONE, UP) or agent.detect(REDSTONE, DOWN):  
    agent.move(BACK, 2)  
    agent.turn(RIGHT_TURN)  
    agent.move(FORWARD, 2)  
agent.move(FORWARD, 1)
```

Fixed code:

Why was it wrong?

Bug C — Rule #4 is `RS LEFT and RS RIGHT → up 5` . This has the AND right but the wrong move — it turns instead of climbing.

```
if agent.detect(REDSTONE, LEFT) and agent.detect(REDSTONE, RIGHT):  
    agent.turn(RIGHT_TURN)  
agent.move(FORWARD, 1)
```

Fixed code:

Why was it wrong?

4 · Write the Missing Rules 🖋️

Rule #2 is `RS RIGHT and RS DOWN → move up 1, turn left` . Write the full `elif` branch.

Rule #5 is `RS UP and RS DOWN → move back 2, turn right, move forward 2` . Write the full `elif` branch.

Add the single checks too. Write the `elif` for a lone `RS LEFT` (turn right) and a lone `RS RIGHT` (turn left). One line: why do these go *after* all the AND checks?

5 · Make It Your Own

Open the **Week 3** world and find the hardest maze from the picture. Start your agent at its entrance, then run your solver. Change **one thing**, predict, then test.

Pick **one** (or your own):

- Add a `back` command that turns the agent around (two right turns).
- Make the agent `agent.place(...)` a block each time an AND rule fires, so you can see every rule it hit.
- Change rule #4 from `UP, 5` to a new number and watch where it lands.

Which change? Write the code.

Predict: what will happen?

Test it. What actually happened? If it surprised you, why?

6 · Finish the Maze 📷

Run your solver and get the agent to the **diamond block** goal. See where it sticks, fix a rule, run again. Send a photo or video of the agent at the goal, then explain. Write 4 to 6 sentences.

My solver used **AND** because ...

The rule that reads two redstones at once is ...

A tile with only one redstone made the agent ...

The change I made in section 5 was ...

The hardest rule to get right was ...

If I had more time, I would ...

7 · Record Your Walkthrough 🎥

Film the agent solving the maze on your phone. Teach it like the viewer has never seen it. Try these words: **detect**, **redstone**, **AND**, **both**, **rule**, **loop**.

1. Show the redstone in the maze before you start.
2. Type `run` and show the agent following your rules.
3. Read one `if ... and ...` rule out loud and say what makes it fire.
4. Show one tile where only one redstone was there, so no AND rule ran.

Write what you will say. Plan it here first — read from it while filming.

Submit ✓

Send this worksheet + your walkthrough video to teacher on KakaoTalk.